Secrets Management

Contents

Secrets Management **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

Secrets Management

Scope

Where credentials live, how they are loaded, and what happens when one leaks.

Official requirements

Subject V.8, verbatim in effect:

> All credentials, API keys, environment variables etc. must be saved locally in a `.env` file and ignored by git. Publicly stored credentials will lead directly to project failure.

This is the single rule in the subject with a stated automatic-failure consequence.

Project policy

Storage

- Every secret is read from the environment. No secret appears in source, in `appsettings.json`, in a test fixture, in a migration, in a Dockerfile, or in a comment.
- `.env` is git-ignored from the first commit. `.gitignore` covers `.env`, `.env.*`, `*.pem`, `*.key`.
- `.env.example` **is** committed: every key present, every value a placeholder. It is the contract for what a fresh clone needs.
- `appsettings.json` holds non-secret configuration only and may be read by anyone.
- For local development, .NET user-secrets is acceptable as an alternative to `.env`; it must never be the only documented path, since the subject names `.env`.

Keys this project will hold

Key	Holder
`YOUTUBE_API_KEY`	backend only
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`	secret is backend only
`FACEBOOK_APP_ID` / `FACEBOOK_APP_SECRET`	secret is backend only

Key	Holder
`JWT_SIGNING_KEY`	backend only
`DATABASE_URL` (pooled, or the local container)	backend only
`DATABASE_DIRECT_URL` (migrations)	backend only, CI only
`SUPABASE_SERVICE_ROLE_KEY` (only if ever issued)	backend only — never a client
`MAIL_PROVIDER`, `MAIL_API_KEY`, `MAIL_FROM`	backend only

Variable names are **vendor-neutral**: the application reads `DATABASE_URL`, not a name containing a provider. Which database it points at is a deployment concern (`brain/decisions/ADR-0026-environment-bring-up.md`).

Credentials are created **just in time**, immediately before the slice that needs them. Fewer keys exist earlier, so there is less to leak while the design is still moving.

No client ever holds a database connection string or a Supabase key. The mobile app reaches the database only through our API.

A key shipped inside a mobile app is public — a bundle can be unpacked. The mobile client never holds the YouTube API key or any OAuth client secret.

Handling

- Never `echo`, print, or log a secret value, including during debugging.
- Never paste a secret into an issue, a pull request, a chat message, or an AI prompt.
- Secrets are not stored in the Brain, in `docs/`, or in any note.

Leak response

A secret that reaches any branch, even unpushed, is compromised. In this order:

1. **Rotate the credential at the provider.** Immediately, before anything else.
2. Remove it from the working tree and from history (`git filter-repo` or equivalent).
3. Force-push the rewritten history and tell every team member to re-clone.
4. Record the incident in `brain/lessons/lessons-index.md` — the fact, not the value.

Deleting the file in a new commit does not fix anything. The value stays in history and is already compromised.

Checklist

- ☐ `.env` git-ignored, verified with `git check-ignore -v .env`
- ☐ `.env.example` complete and placeholder-only
- ☐ `git log -p` and the working tree contain no real key
- ☐ No secret in the mobile bundle
- ☐ Every team member knows the rotate-first leak procedure

References

- Project subject, section V.8
- OWASP Secrets Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Ch
- Twelve-Factor config — https://12factor.net/config

Related

`../application-security/STANDARD.md` · `../../00-foundations/git-workflow/STANDARD.md`